Phase 1: Exploratory Data Analysis & Data Alignment

In [1]:
```
import os
import pandas as pd
import numpy as np

base_dir = "/home/zibo127/Documents/institution_subnets"

df_10m = pd.read_csv(os.path.join(base_dir, "agg_10_minutes", "0.csv"))
df_1h = pd.read_csv(os.path.join(base_dir, "agg_1_hour", "0.csv" ))

print("Raw 10m shape:", df_10m.shape)
print("Raw Hourly shape:", df_1h.shape)
print("Columns:", list(df_10m.columns))
```

```
Raw 10m shape: (40296, 19)
Raw Hourly shape: (6717, 19)
Columns: ['id_time', 'n_flows', 'n_packets', 'n_bytes', 'sum_n_dest_asn', 'a
verage_n_dest_asn', 'std_n_dest_asn', 'sum_n_dest_ports', 'average_n_dest_po
rts', 'std_n_dest_ports', 'sum_n_dest_ip', 'average_n_dest_ip', 'std_n_dest_
ip', 'tcp_udp_ratio_packets', 'tcp_udp_ratio_bytes', 'dir_ratio_packets', 'd
ir_ratio_bytes', 'avg_duration', 'avg_ttl']
```

In [2]:
```
idx_10m = pd.Index(np.arange(df_10m["id_time"].min(), df_10m["id_time"].max(
df_10m_clean = df_10m.set_index("id_time").reindex(idx_10m).interpolate(meth

idx_1h = pd.Index(np.arange(df_1h["id_time"].min(), df_1h["id_time"].max() +
df_1h_clean = df_1h.set_index("id_time").reindex(idx_1h).interpolate(method=

df_10m_hourly = df_10m_clean.groupby(df_10m_clean.index // 6).agg({
    "n_flows": "sum",
    "n_packets": "sum",
    "n_bytes": "sum"
})

common_hours = df_10m_hourly.index.intersection(df_1h_clean.index)
flows_resampled = df_10m_hourly.loc[common_hours, "n_flows"]
flows_actual = df_1h_clean.loc[common_hours, "n_flows"]

correlation = np.corrcoef(flows_resampled, flows_actual)[0, 1]
mape = np.mean(np.abs(flows_resampled - flows_actual) / (flows_actual + 1))

print(f"Naive Correlation: {correlation:.6f}")
print(f"Naive MAPE: {mape:.6f}%")

all_comparison = pd.DataFrame({
    'Calculated_10m_Sum': flows_resampled,
    'Actual_Hourly': flows_actual
})
all_comparison['Difference'] = all_comparison['Calculated_10m_Sum'] - all_cc
```

```
mismatches = all_comparison[all_comparison['Difference'] != 0]
print(f"Total mismatched hours: {len(mismatches)}")
```

```
Naive Correlation: 0.924825
Naive MAPE: 13.299313%
Total mismatched hours: 6237
```

In [3]:
```
print(all_comparison.loc[475:485])
```

```
         Calculated_10m_Sum  Actual_Hourly  Difference
id_time
475                360962.0       360962.0         0.0
476                271958.0       271958.0         0.0
477                208291.0       208291.0         0.0
478                173958.0       173958.0         0.0
479                140797.0       140797.0         0.0
480                139041.0       139919.0      -878.0
481                116901.0       139041.0    -22140.0
482                117498.0       116901.0       597.0
483                128621.0       117498.0     11123.0
484                158740.0       128621.0     30119.0
485                199598.0       158740.0     40858.0
```

In [4]:
```
df_1h_shifted = df_1h.copy()
df_1h_shifted.loc[df_1h_shifted['id_time'] >= 481, 'id_time'] -= 1

idx_1h_shifted = pd.Index(np.arange(df_1h_shifted["id_time"].min(), df_1h_sh
df_1h_clean_shifted = df_1h_shifted.set_index("id_time").reindex(idx_1h_shif

common_hours_shifted = df_10m_hourly.index.intersection(df_1h_clean_shifted.
flows_resampled_s = df_10m_hourly.loc[common_hours_shifted, "n_flows"]
flows_actual_s = df_1h_clean_shifted.loc[common_hours_shifted, "n_flows"]

corr_s = np.corrcoef(flows_resampled_s, flows_actual_s)[0, 1]
mape_s = np.mean(np.abs(flows_resampled_s - flows_actual_s) / (flows_actual_

all_comp_s = pd.DataFrame({
    'Calculated_10m_Sum': flows_resampled_s,
    'Actual_Hourly': flows_actual_s
})
all_comp_s['Difference'] = all_comp_s['Calculated_10m_Sum'] - all_comp_s['Ac
mismatches_s = all_comp_s[all_comp_s['Difference'] != 0]

print(f"Shifted Correlation: {corr_s:.6f}")
print(f"Shifted MAPE: {mape_s:.6f}%")
print(f"Remaining mismatched hours: {len(mismatches_s)}")
print(mismatches_s.head(10))
```

```
Shifted Correlation: 0.998156
Shifted MAPE: 1.653818%
Remaining mismatched hours: 4282
         Calculated_10m_Sum  Actual_Hourly  Difference
id_time
2435                241186.0         201081     40105.0
2436                238955.0         238052       903.0
2437                301213.0         285974     15239.0
2438                253125.0         266439    -13314.0
2439                257608.0         258968     -1360.0
2440                259460.0         255670      3790.0
2441                291822.0         285970      5852.0
2442                301433.0         302770     -1337.0
2443                288803.0         291537     -2734.0
2444                309341.0         306079      3262.0
```

In [5]:
```
expected_hourly = set(range(df_1h["id_time"].min(), df_1h["id_time"].max() +
actual_hourly = set(df_1h["id_time"])
missing_hourly = sorted(list(expected_hourly - actual_hourly))

expected_10m = set(range(df_10m["id_time"].min(), df_10m["id_time"].max() +
actual_10m = set(df_10m["id_time"])
missing_10m = sorted(list(expected_10m - actual_10m))

print("Missing hourly indices:", missing_hourly)
print("Missing 10-minute indices:", missing_10m)
```

```
Missing hourly indices: [480]
Missing 10-minute indices: [26028, 26029]
```

In [6]:
```
def get_clean_subnet_data(subnet_id: int):
    file_path = os.path.join(base_dir, "agg_10_minutes", f"{subnet_id}.csv")
    df = pd.read_csv(file_path)

    t_min = df["id_time"].min()
    t_max = df["id_time"].max()
    idx = pd.Index(np.arange(t_min, t_max + 1), name="id_time")
    df_clean = df.set_index("id_time").reindex(idx).interpolate(method="line

    agg_dict = {
        "n_flows": "sum",
        "n_packets": "sum",
        "n_bytes": "sum",
        "avg_duration": "mean",
        "avg_ttl": "mean",
        "tcp_udp_ratio_packets": "mean",
        "tcp_udp_ratio_bytes": "mean",
        "dir_ratio_packets": "mean",
        "dir_ratio_bytes": "mean",
        "average_n_dest_asn": "mean",
        "average_n_dest_ports": "mean",
        "average_n_dest_ip": "mean"
    }

    agg_dict = {col: agg for col, agg in agg_dict.items() if col in df_clear
    df_hourly = df_clean.groupby(df_clean.index // 6).agg(agg_dict)
```

```
    df_hourly.index.name = "id_time"
    return df_hourly

df_hour_clean = get_clean_subnet_data(0)
print("Clean Hourly Shape:", df_hour_clean.shape)
```

```
Clean Hourly Shape: (6717, 12)
```

In [7]:
```
flows_skew_orig = df_hour_clean['n_flows'].skew()
flows_kurt_orig = df_hour_clean['n_flows'].kurt()

bytes_skew_orig = df_hour_clean['n_bytes'].skew()
bytes_kurt_orig = df_hour_clean['n_bytes'].kurt()

flows_log = np.log1p(df_hour_clean['n_flows'])
bytes_log = np.log1p(df_hour_clean['n_bytes'])

print(f"Original flows skewness: {flows_skew_orig:.4f}, kurtosis: {flows_kur
print(f"Log1p flows skewness: {flows_log.skew():.4f}, kurtosis: {flows_log.k

print(f"Original bytes skewness: {bytes_skew_orig:.4f}, kurtosis: {bytes_kur
print(f"Log1p bytes skewness: {bytes_log.skew():.4f}, kurtosis: {bytes_log.k
```

```
Original flows skewness: 0.7516, kurtosis: 0.4459
Log1p flows skewness: -0.3774, kurtosis: -0.2051

Original bytes skewness: 0.9322, kurtosis: 0.6346
Log1p bytes skewness: -0.5497, kurtosis: -0.0783
```

In [8]:
```
Q1_raw = df_hour_clean['n_flows'].quantile(0.25)
Q3_raw = df_hour_clean['n_flows'].quantile(0.75)
IQR_raw = Q3_raw - Q1_raw
upper_raw = Q3_raw + 1.5 * IQR_raw
outliers_iqr_raw = (df_hour_clean['n_flows'] > upper_raw).sum()

z_raw = (df_hour_clean['n_flows'] - df_hour_clean['n_flows'].mean()) / df_hc
outliers_z_raw = (z_raw.abs() > 3).sum()

Q1_log = flows_log.quantile(0.25)
Q3_log = flows_log.quantile(0.75)
IQR_log = Q3_log - Q1_log
lower_log = Q1_log - 1.5 * IQR_log
upper_log = Q3_log + 1.5 * IQR_log
outliers_iqr_log = ((flows_log < lower_log) | (flows_log > upper_log)).sum()

z_log = (flows_log - flows_log.mean()) / flows_log.std()
outliers_z_log = (z_log.abs() > 3).sum()

print("IQR Outliers (Raw):", outliers_iqr_raw)
print("Z-score Outliers (Raw):", outliers_z_raw)
print("IQR Outliers (Log1p):", outliers_iqr_log)
print("Z-score Outliers (Log1p):", outliers_z_log)
```

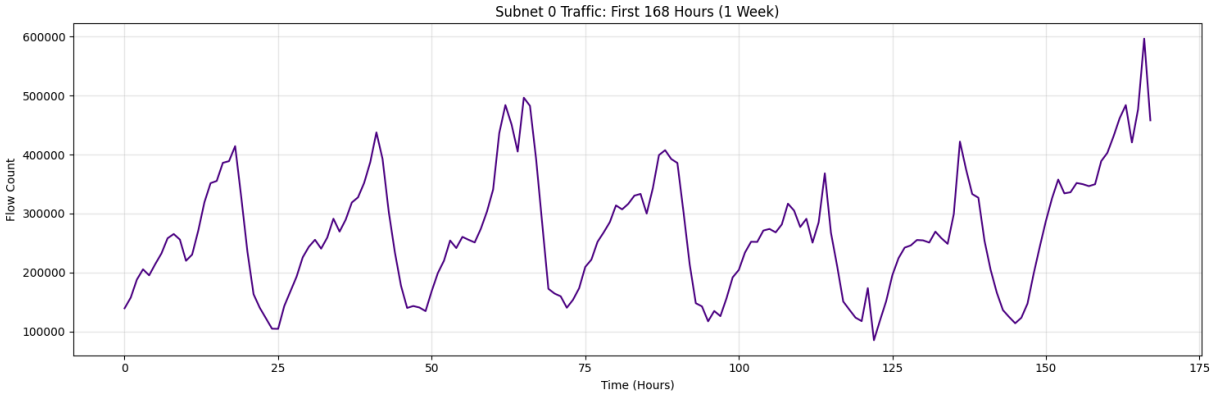
```
IQR Outliers (Raw):
```

```
 52
Z-score Outliers (Raw): 36
IQR Outliers (Log1p): 22
Z-score Outliers (Log1p): 13
```

In [9]:
```
corr_matrix = df_hour_clean.corr(method='pearson')
print(corr_matrix[['n_flows', 'n_packets', 'n_bytes']])
```

```
                         n_flows  n_packets   n_bytes
n_flows                 1.000000   0.913812  0.904820
n_packets               0.913812   1.000000  0.997951
n_bytes                 0.904820   0.997951  1.000000
avg_duration            0.630777   0.712530  0.713994
avg_ttl                -0.108972  -0.125491 -0.138895
tcp_udp_ratio_packets   0.362052   0.354316  0.339085
tcp_udp_ratio_bytes     0.390493   0.379999  0.365122
dir_ratio_packets       0.155328   0.129867  0.113759
dir_ratio_bytes        -0.186071  -0.222571 -0.235331
average_n_dest_asn      0.806969   0.786738  0.781627
average_n_dest_ports    0.422920   0.193765  0.191514
average_n_dest_ip       0.853706   0.791032  0.789526
```

In [10]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(15, 5))
plt.plot(df_hour_clean.index[:168], df_hour_clean['n_flows'].iloc[:168], col
plt.title('Subnet 0 Traffic: First 168 Hours (1 Week)')
plt.xlabel('Time (Hours)')
plt.ylabel('Flow Count')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



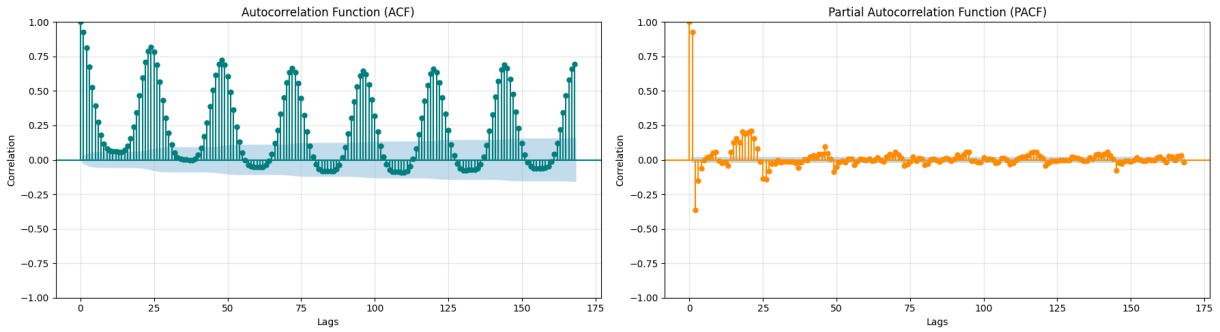
Phase 2: Time Series Foundations

In [11]:
```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df_hour_clean['n_flows'].dropna())
print(f"ADF Statistic: {result[0]:.6f}")
print(f"p-value: {result[1]:.6e}")
print("Critical Values:")
```

```
for key, value in result[4].items():
    print(f"\t{key}: {value:.3f}")
```

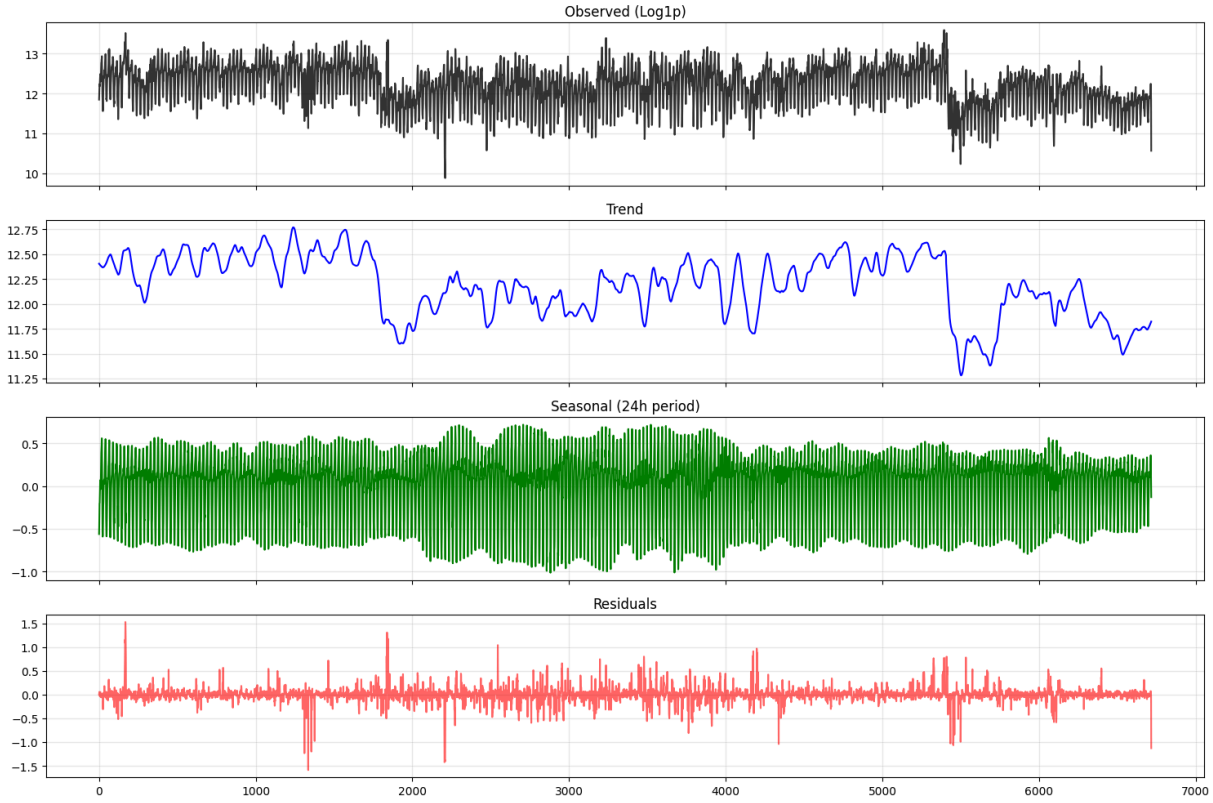
```
ADF Statistic: -5.296685
p-value: 5.562890e-06
Critical Values:
        1%: -3.431
        5%: -2.862
        10%: -2.567
```

In [12]:
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(18, 5))

plot_acf(df_hour_clean['n_flows'].dropna(), lags=168, ax=axes[0], color='tea
axes[0].set_title('Autocorrelation Function (ACF)')
axes[0].set_xlabel('Lags')
axes[0].set_ylabel('Correlation')
axes[0].grid(True, alpha=0.3)

plot_pacf(df_hour_clean['n_flows'].dropna(), lags=168, ax=axes[1], color='da
axes[1].set_title('Partial Autocorrelation Function (PACF)')
axes[1].set_xlabel('Lags')
axes[1].set_ylabel('Correlation')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



In [13]:
```
from statsmodels.tsa.seasonal import STL

stl_log = STL(np.log1p(df_hour_clean['n_flows']), period=24, robust=True).fi

fig, axes = plt.subplots(4, 1, figsize=(15, 10), sharex=True)

axes[0].plot(np.log1p(df_hour_clean['n_flows']), color='black', alpha=0.8)
axes[0].set_title('Observed (Log1p)')
axes[0].grid(True, alpha=0.3)

axes[1].plot(stl_log.trend, color='blue')
axes[1].set_title('Trend')
axes[1].grid(True, alpha=0.3)

axes[2].plot(stl_log.seasonal, color='green')
axes[2].set_title('Seasonal (24h period)')
axes[2].grid(True, alpha=0.3)
```

```
axes[3].plot(stl_log.resid, color='red', alpha=0.6)
axes[3].set_title('Residuals')
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Phase 3: Time Series Feature Engineering

In [14]:
```
df_features = df_hour_clean.copy()

df_features['lag_1'] = df_features['n_flows'].shift(1)
df_features['lag_2'] = df_features['n_flows'].shift(2)
df_features['lag_24'] = df_features['n_flows'].shift(24)
df_features['lag_168'] = df_features['n_flows'].shift(168)

df_features['rolling_mean_6h'] = df_features['n_flows'].shift(1).rolling(wir
df_features['rolling_std_6h'] = df_features['n_flows'].shift(1).rolling(winc

df_features['rolling_mean_24h'] = df_features['n_flows'].shift(1).rolling(wi
df_features['rolling_std_24h'] = df_features['n_flows'].shift(1).rolling(wir

df_features['hour_of_day'] = df_features.index % 24
df_features['day_of_week'] = (df_features.index // 24) % 7
df_features['is_weekend'] = df_features['day_of_week'].isin([5, 6]).astype(i

print("Feature Matrix shape:", df_features.shape)
print(df_features[['n_flows', 'lag_1', 'rolling_mean_24h', 'is_weekend']].dr
```

```
Feature Matrix shape: (6717, 23)
          n_flows     lag_1  rolling_mean_24h  is_weekend
id_time
24       104858.0  122737.0     251553.166667           0
25       104630.0  104858.0     250112.416667           0
26       143362.0  104630.0     247908.041667           0
27       168138.0  143362.0     246047.916667           0
28       192996.0  168138.0     244491.583333           0
```

Phase 4: Forecasting & Model Evaluation

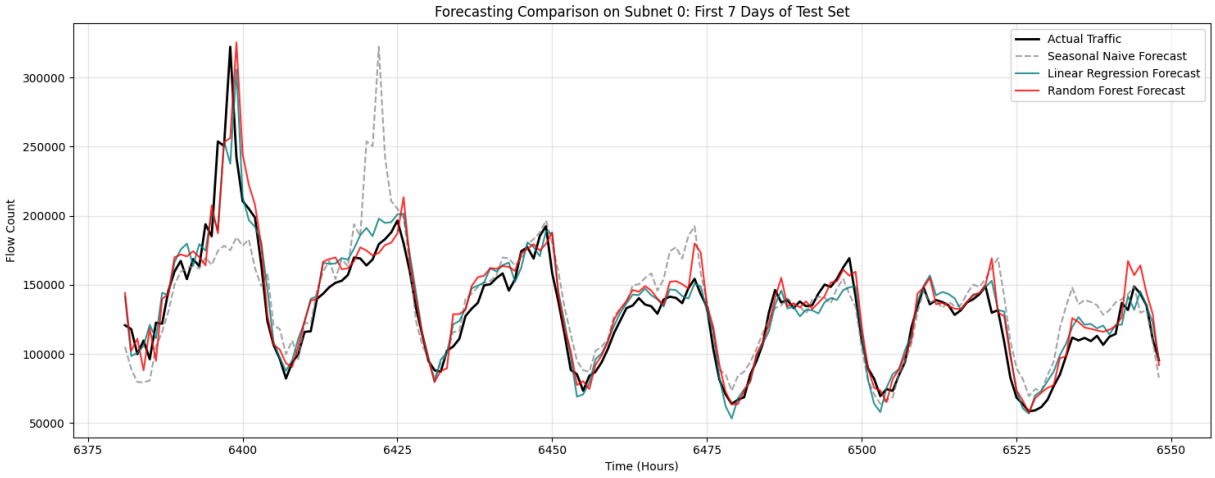
In [15]:
```
df_ml = df_features.dropna().copy()

target_col = 'n_flows'
feature_cols = [
    'lag_1', 'lag_2', 'lag_24', 'lag_168',
    'rolling_mean_6h', 'rolling_std_6h',
    'rolling_mean_24h', 'rolling_std_24h',
    'hour_of_day', 'day_of_week', 'is_weekend'
]

X = df_ml[feature_cols]
y = df_ml[target_col]

test_hours = 336
X_train, X_test = X.iloc[:-test_hours], X.iloc[-test_hours:]
y_train, y_test = y.iloc[:-test_hours], y.iloc[-test_hours:]

print("Train Features Shape:", X_train.shape)
print("Test Features Shape:", X_test.shape)
print("Train Index Range:", X_train.index.min(), "to", X_train.index.max())
print("Test Index Range:", X_test.index.min(), "to", X_test.index.max())
```

```
Train Features Shape: (6213, 11)
Test Features Shape: (336, 11)
Train Index Range: 168 to 6380
Test Index Range: 6381 to 6716
```

In [16]:
```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor

y_pred_baseline = X_test['lag_24']

lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)

rf_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

def print_metrics(model_name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
```

```
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f"[{model_name}] MAE: {mae:.2f} | RMSE: {rmse:.2f} | R2: {r2:.4f}"

print_metrics("Seasonal Naive Baseline", y_test, y_pred_baseline)
print_metrics("Linear Regression", y_test, y_pred_lr)
print_metrics("Random Forest", y_test, y_pred_rf)

plot_hours = 168
plt.figure(figsize=(15, 6))
plt.plot(y_test.index[:plot_hours], y_test.iloc[:plot_hours], label='Actual
plt.plot(y_test.index[:plot_hours], y_pred_baseline.iloc[:plot_hours], label
plt.plot(y_test.index[:plot_hours], y_pred_lr[:plot_hours], label='Linear Re
plt.plot(y_test.index[:plot_hours], y_pred_rf[:plot_hours], label='Random Fo

plt.title('Forecasting Comparison on Subnet 0: First 7 Days of Test Set')
plt.xlabel('Time (Hours)')
plt.ylabel('Flow Count')
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```

```
[Seasonal Naive Baseline] MAE: 14326.97 | RMSE: 21780.15 | R2: 0.6322
[Linear Regression] MAE: 9239.07 | RMSE: 13273.85 | R2: 0.8634
[Random Forest] MAE: 10209.24 | RMSE: 14862.84 | R2: 0.8287
```

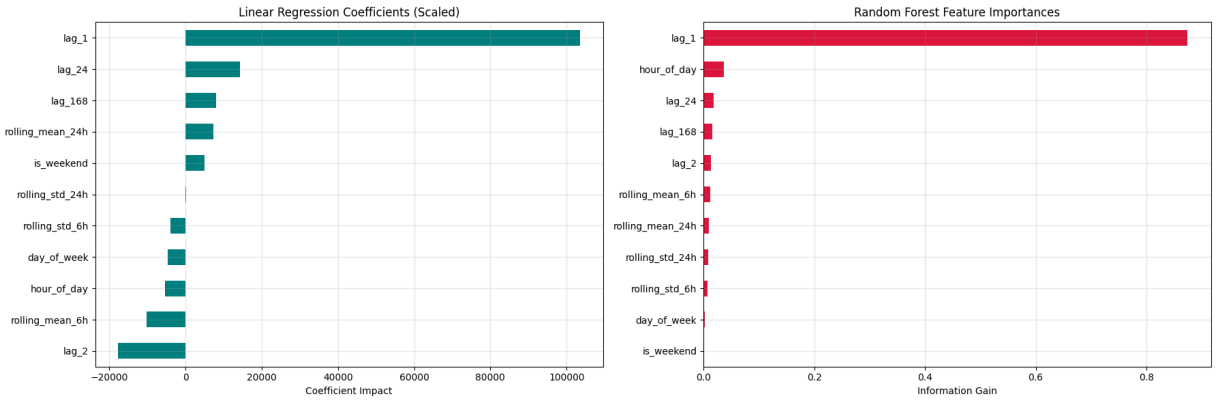


Phase 5: Model Interpretation

In [17]:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
lr_scaled = LinearRegression()
lr_scaled.fit(X_train_scaled, y_train)

lr_coefs = pd.Series(lr_scaled.coef_, index=feature_cols).sort_values(ascenc
rf_importances = pd.Series(rf_model.feature_importances_, index=feature_cols

fig, axes = plt.subplots(1, 2, figsize=(18, 6))
```

```
lr_coefs.plot(kind='barh', ax=axes[0], color='teal')
axes[0].set_title('Linear Regression Coefficients (Scaled)')
axes[0].set_xlabel('Coefficient Impact')
axes[0].grid(True, alpha=0.3)

rf_importances.plot(kind='barh', ax=axes[1], color='crimson')
axes[1].set_title('Random Forest Feature Importances')
axes[1].set_xlabel('Information Gain')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Phase 6: Unsupervised Learning (Subnet Clustering)

In [18]:

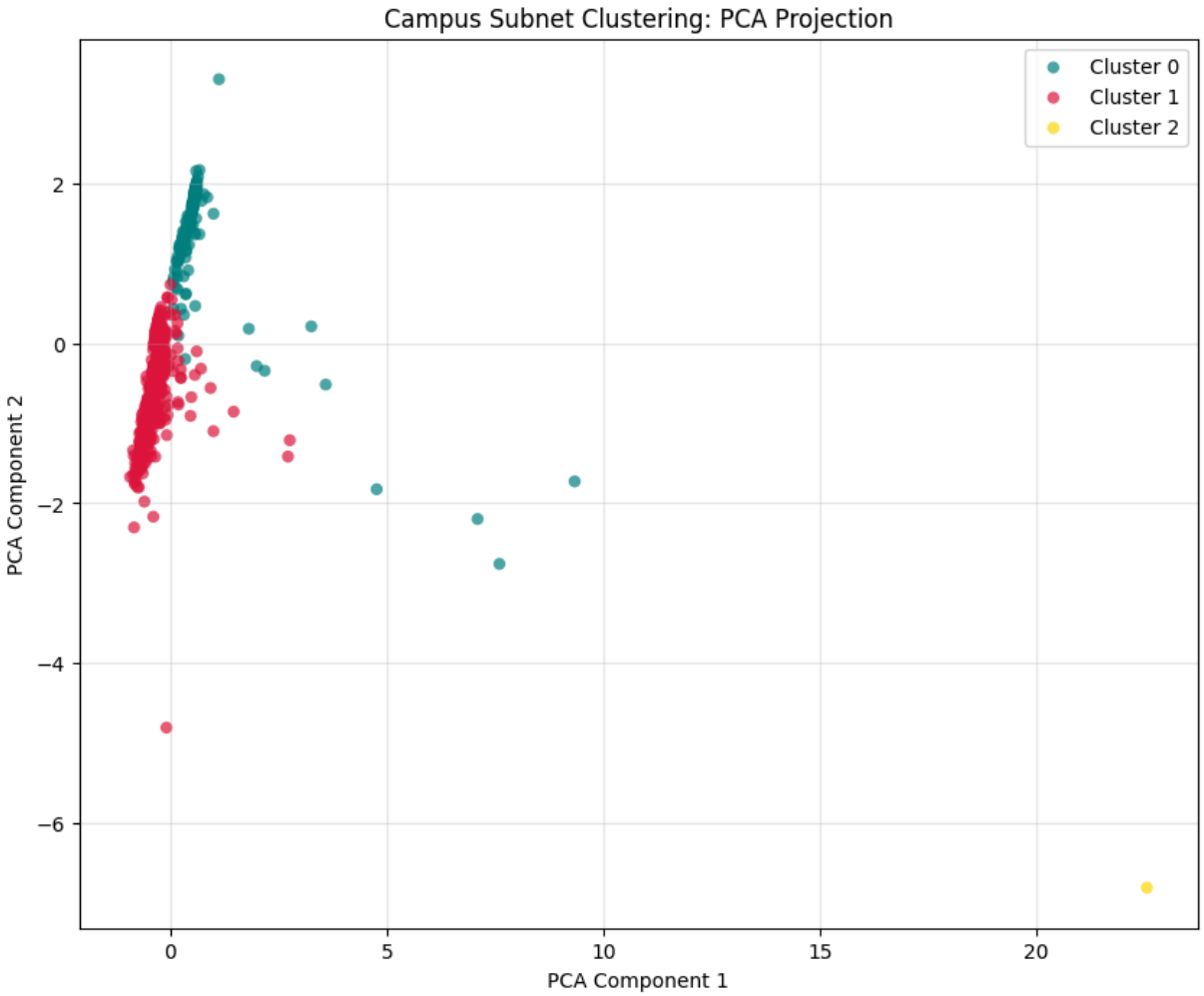
```
import glob

subnets_data = []
hourly_files = glob.glob(os.path.join(base_dir, "agg_1_hour", "*.csv"))

print(f"Processing {len(hourly_files)} files...")

for file_path in sorted(hourly_files):
    subnet_id = int(os.path.basename(file_path).split('.')[0])
    df_sub = pd.read_csv(file_path)

    idx_sub = pd.Index(np.arange(df_sub["id_time"].min(), df_sub["id_time"].
    df_sub_clean = df_sub.set_index("id_time").reindex(idx_sub).interpolate(

    hour_of_day = df_sub_clean.index % 24
    day_of_week = (df_sub_clean.index // 24) % 7
    is_weekend = day_of_week.isin([5, 6])

    mean_flows = df_sub_clean['n_flows'].mean()
    std_flows = df_sub_clean['n_flows'].std()

    weekday_mean = df_sub_clean.loc[~is_weekend, 'n_flows'].mean()
    weekend_mean = df_sub_clean.loc[is_weekend, 'n_flows'].mean()
    weekend_ratio = weekend_mean / (weekday_mean + 1e-5)
```

```
    is_daytime = hour_of_day.isin(range(8, 20))
    day_mean = df_sub_clean.loc[is_daytime, 'n_flows'].mean()
    night_mean = df_sub_clean.loc[~is_daytime, 'n_flows'].mean()
    day_night_ratio = day_mean / (night_mean + 1e-5)

    hourly_profile = df_sub_clean.groupby(df_sub_clean.index % 24)['n_flows'
    peak_hour = hourly_profile.idxmax()

    subnets_data.append({
        'subnet_id': subnet_id,
        'mean_flows': mean_flows,
        'std_flows': std_flows,
        'weekend_ratio': weekend_ratio,
        'day_night_ratio': day_night_ratio,
        'peak_hour': peak_hour
    })

df_fingerprints = pd.DataFrame(subnets_data).set_index('subnet_id')
print("Feature matrix shape:", df_fingerprints.shape)
print(df_fingerprints.head())
```

```
Processing 546 files...
Feature matrix shape: (546, 5)
              mean_flows      std_flows  weekend_ratio  day_night_ratio  \
subnet_id
0          221716.507294  104989.043241       0.874415         1.530393
1              32.528878      49.050060       0.674604         1.335482
10           7337.162176    6771.400165       0.437425         1.494423
100          8818.739580    4596.235046       0.573494         1.326100
101           217.251191     174.962190       1.077416         0.897970

            peak_hour
subnet_id
0                  19
1                  10
10                  9
100                 7
101                23
```

In [19]:
```
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

df_fingerprints_clean = df_fingerprints.fillna(0).replace([np.inf, -np.inf],

scaler_f = StandardScaler()
X_scaled = scaler_f.fit_transform(df_fingerprints_clean)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
df_fingerprints_clean['cluster'] = clusters

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
df_fingerprints_clean['pca_1'] = X_pca[:, 0]
df_fingerprints_clean['pca_2'] = X_pca[:, 1]
```

```
plt.figure(figsize=(10, 8))
colors = ['teal', 'crimson', 'gold']
for cluster_id in range(3):
    sub_cluster = df_fingerprints_clean[df_fingerprints_clean['cluster'] ==
    plt.scatter(sub_cluster['pca_1'], sub_cluster['pca_2'],
                label=f'Cluster {cluster_id}', color=colors[cluster_id], alp

plt.title('Campus Subnet Clustering: PCA Projection')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(df_fingerprints_clean.groupby('cluster')[['mean_flows', 'std_flows', '
```

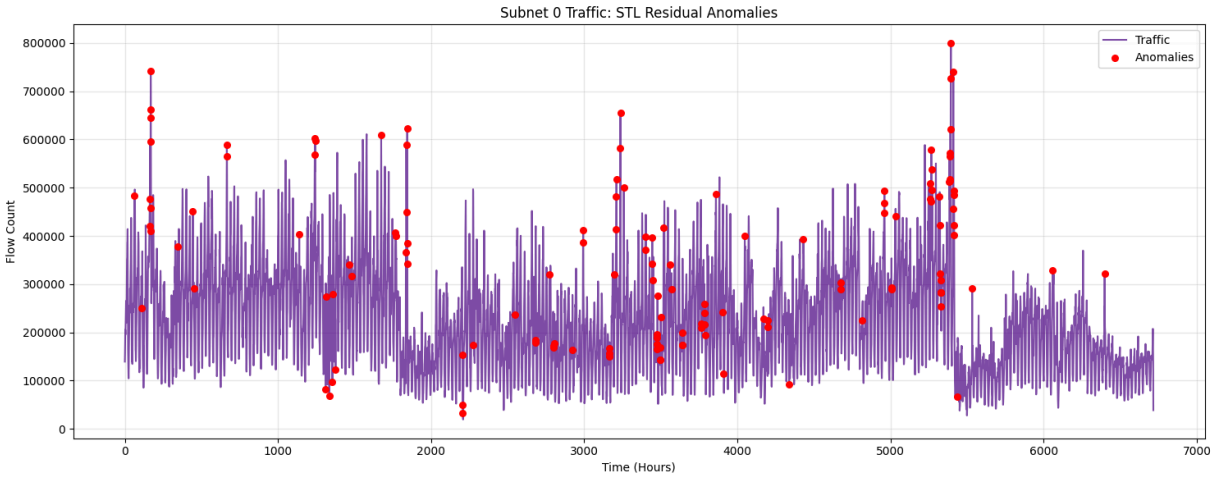


```
            mean_flows     std_flows  weekend_ratio  day_night_ratio  \
cluster
0          32186.376627  1.421064e+04       1.067814         0.978701
1           9789.342390  6.065240e+03       0.703082         1.068475
2         970666.308276  4.281238e+06       1.016434         0.914341

           peak_hour
cluster
0          19.853503
1           5.469072
2          21.000000
```

Phase 7: Anomaly Detection

In [20]:
```
from statsmodels.tsa.seasonal import STL

stl = STL(df_hour_clean['n_flows'], period=24, robust=True).fit()
residuals = stl.resid

res_mean = residuals.mean()
res_std = residuals.std()
z_scores = (residuals - res_mean) / res_std

anomalies = df_hour_clean[z_scores.abs() > 3]
print(f"Total anomalies detected: {len(anomalies)} ({len(anomalies)/len(df_h

plt.figure(figsize=(15, 6))
plt.plot(df_hour_clean.index, df_hour_clean['n_flows'], color='indigo', alph
plt.scatter(anomalies.index, anomalies['n_flows'], color='red', label='Anoma
plt.title('Subnet 0 Traffic: STL Residual Anomalies')
plt.xlabel('Time (Hours)')
plt.ylabel('Flow Count')
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(anomalies[['n_flows']].head(10))
```

```
Total anomalies detected: 136 (2.02%)
```



id_time	n_flows
62	483761.0
112	250673.0
164	420437.0
165	476590.0
166	596425.0
167	457935.0
168	645086.0
169	741350.0
170	661432.0
171	410264.0